

RAPPORT D'ANALYSE

Projet audité : MusiQuali (« Musi-quali »)

SAÉ S3 – Diffusion sonore multi-site

Équipe auditée : D. Boylan, D. Cheblal, L. Hoarau, A. Mastouri, A. Shishkovskiy-Ryschenkow, A. Talbi

Audit réalisé par l'équipe SoundStream (Synapse Audio)

Préambule et méthodologie

Dans le cadre de la reprise des projets du premier semestre, chaque groupe analyse le travail d'une autre équipe afin de décider quelle base reprendre pour la suite de la SAÉ. Le présent rapport évalue le projet MusiQuali selon les cinq critères convenus : (1) fonctionnalité et respect du besoin, (2) qualité du code, (3) gestion des données, (4) interface utilisateur, (5) installation et déploiement.

Comme pour les autres audits, le dépôt fourni ne contient pas les documents de cadrage propres à MusiQuali (user stories, cahier des charges, MEA). Nous évaluons donc la fonctionnalité par rapport au besoin global exprimé par notre client commun, M. Patrice Bringuier, et à la consigne « diffusion sonore » : système multi-site à gestion centralisée, supervision de l'état des lecteurs (UP/KO) par ICMP, synchronisation de la playlist locale, continuité de service en cas de coupure réseau, consignation des diffusions et messages urgents prioritaires.

Précision importante : cet audit se veut constructif. MusiQuali est, sur plusieurs points, un projet sérieux et plus abouti que d'autres dépôts analysés, notamment parce qu'il implémente une vraie communication réseau. Chaque critère se termine donc par des pistes d'amélioration concrètes.

Critère 1 — Fonctionnalité et respect du besoin

Ce qui fonctionne et constitue un vrai point fort

Contrairement à plusieurs autres projets, MusiQuali a réellement traité la dimension réseau, qui est le cœur de cette SAÉ. Le serveur central gère une table de Raspberry (ajout, suppression, sélection par identifiant) et pousse effectivement les fichiers vers chaque lecteur distant via rsync et SSH (RaspberryService, RaspberryController). L'identifiant du lecteur n'est pas figé en dur : on peut déclarer plusieurs Raspberry par leur IP.

- **Continuité hors-ligne pensée par architecture** : le script RAS.py tourne en local sur chaque Raspberry, lit le planning synchronisé (planning_export.json) et joue les fichiers MP3 stockés localement via pygame. La logique de « push » depuis le serveur permet donc, en principe, de continuer à diffuser même après une coupure réseau exactement l'esprit demandé par le client.

- **Authentification et rôles** : la connexion fonctionne avec trois rôles (admin, marketing, commercial) et un décorateur reqrole bien pensé qui protège les routes. C'est propre et conforme au besoin.
- **Gestion de contenu** : création/suppression de playlists, upload de MP3 avec lecture de la durée réelle (mutagen) et planning hebdomadaire éditable en glisser-déposer. L'ensemble est fonctionnel pour la partie marketing.

Fonctionnalités attendues mais absentes ou non opérationnelles

- **Messages urgents / interruption** : c'est la lacune la plus importante. Le besoin client prévoit qu'un message commercial urgent coupe la musique le temps de sa diffusion (logique « canal 0 / canal 1 » décrite dans la note de fonctionnement). Or RAS.py n'implémente qu'un seul lecteur pygame qui joue la playlist du jour une seule fois ; il n'existe aucun mécanisme d'interruption, ni de second canal, ni de prise en compte d'un fichier de messages mis à jour à la volée. Côté interface, l'espace commercial (bouton lecture, recherche) n'est relié à aucune action.
- **Consignation centrale des diffusions** : RAS.py écrit bien un journal local (« played ... at ... ») sur chaque Raspberry, mais rien ne remonte ces logs vers la base centrale. Une classe LogsDAO existe mais n'est jamais instanciée, et le bouton « Télécharger les logs » de l'interface admin ne correspond à aucune route. L'historique demandé par le client n'est donc pas exploitable côté serveur.
- **Supervision UP/KO** : le ping existe (ping -c 4 sur action manuelle), mais l'état des lecteurs n'est ni stocké, ni affiché en continu, ni transformé en alerte. Le modèle Raspberry ne comporte pas de champ « état ». La supervision reste donc une action ponctuelle, pas un tableau de bord.
- **Ambiance matin/après-midi** : le besoin d'une bande-son calme le matin et plus énergique l'après-midi n'est pas modélisé ; le planning est une simple liste manuelle de titres.

Note critère 1 : 12/20

Pistes : implémenter l'interruption via un second canal (pygame.mixer.Channel) lisant un dossier « messages » surveillé en continu ; ajouter une route de remontée des logs (rsync inverse ou POST en fin de journée) ; persister l'état UP/KO après chaque ping.

Critère 2 — Qualité du code

L'architecture est l'atout du projet : séparation nette Controllers / Services / DAO / Models, et même une interface UserDAOInterface. Pour un BUT2, l'intention MVC en couches est tout à fait louable et bien au-dessus du minimum attendu. En revanche, l'exécution souffre de beaucoup de désordre.

- **Code dupliqué et fichiers fantômes** : deux classes PlaylistService différentes portant le même nom (service_playlist.py et playlistServiceMarketing.py) ; deux modèles concurrents Music.py et Musique.py (ce dernier inutilisé) ; un modèle Logs présent en double (models/Logs.py et dans LogsDAO.py) jamais branché ; des templates en plusieurs

versions (login.html / login_v2.html, commercial.html, testRasp.html) laissées dans le dépôt.

- **Routes en conflit** : la route /admin est déclarée deux fois (admin_page dans controllers_commercial.py et admin_dashboard dans AdminController.py). De plus add_playlist redirige vers url_for('main.admin_page') alors qu'aucun blueprint « main » n'existe l'appel échouerait. Le module Blueprint est importé mais jamais utilisé.
- **Bugs latents** : MarketingController utilise des variables (musics, selected_playlist_id) avant leur définition dans la branche « commercial » (risque de NameError) ; search_by_title appelle une méthode service.search_by_title qui n'existe pas (le commentaire le reconnaît : « nexiste pas ») ; le script script.js fait document.querySelector('.role_nav').textContent sans vérifier la présence de l'élément, ce qui lève une erreur sur les pages publiques (accueil, login) et casse le reste du JavaScript de la page (mode sombre inclus).
- **Traces de mise au point laissées** : de nombreux print de débogage (« rasp_id : », « rrrrrrr_dao : »), de gros blocs commentés (tout VerifieShell dans RaspberryDAO, add_music, musicsinplaylist), des variables a, b, c, d dans les services, et des commentaires informels (« au final je m'en sers pas mais c utile a savoir lol », « Dragon drop »).
- **Gestion Git fragile** : un fichier de logs (logs[2026-01-22].txt) contient des marqueurs de conflit de fusion non résolus (<<<<<<< HEAD ... ===== ... >>>>>>> testmain) commités tels quels. C'est un indice net d'un usage de Git mal maîtrisé.
- **Absence de tests** : aucun fichier de test n'est présent, alors que le critère mentionne explicitement un code « testé ».

Note critère 2 : 11/20

Pistes : supprimer le code mort et les versions doublons, résoudre le conflit de route /admin, regrouper réellement la configuration de la base en un seul module, et garder un dépôt propre (gitignore, résolution des conflits, commits réguliers).

Critère 3 — Gestion des données

La base SQLite remplit les fonctions de base, et deux bonnes pratiques méritent d'être saluées : les mots de passe sont hachés avec bcrypt, et toutes les requêtes utilisent des paramètres liés (« ? »), ce qui protège des injections SQL — y compris le tri dynamique de MusicDAO, sécurisé par une liste blanche de colonnes. C'est plus rigoureux que la moyenne. Trois problèmes sérieux subsistent néanmoins.

- **Faible de sécurité majeure — base dans un dossier public** : la base est stockée à static/data/database.db (app.static_folder + '/data/database.db'). Elle est donc téléchargeable depuis l'URL /static/data/database.db, comme dans d'autres projets analysés. Même si les mots de passe sont hachés, exposer toute la base (utilisateurs, rôles, structure) reste une faute de fond : une base ne doit jamais être servie publiquement.

À noter aussi un compte par défaut admin/admin créé automatiquement et une clé secrète Flask écrite en dur.

- **Modèle de données non normalisé** : la relation playlist–musique est stockée sous forme d'une chaîne « 1|2|3 » dans une colonne TEXT (music_ids). C'est une violation de la première forme normale (attribut multivaluée), là où une table d'association playlist_musique serait attendue. Par ailleurs la table schedule_item recopie le titre, l'artiste, la durée et le chemin des musiques au lieu de référencer leur identifiant : on a donc de la redondance et un risque d'anomalies de mise à jour (renommer un titre laisse le planning incohérent).
- **Schémas multiples et incohérents** : le module db.py définit des tables (playlists, musics, schedule_days/items) qui ne sont jamais utilisées, tandis que les DAO créent leurs propres tables aux noms différents (playlist, music, schedule_day/schedule_item). Il existe ainsi plusieurs « sources de vérité » pour le même schéma, avec même des valeurs par défaut divergentes (heure de début « 08:00 » dans db.py contre « 00:00 » dans le DAO).
- **Historique non persisté** : comme vu au critère 1, les logs de diffusion ne sont pas stockés en base. La table logs est créée par LogsDAO mais jamais alimentée.

Note critère 3 : 9/20

Pistes : déplacer la base hors de static/ (dossier non servi) ; remplacer music_ids par une table d'association ; faire de schedule_item une simple référence vers music ; unifier la création du schéma dans un seul fichier ; alimenter réellement la table logs.

Critère 4 — Interface utilisateur

Visuellement, l'interface est soignée et propose plusieurs fonctionnalités appréciables qu'on ne retrouve pas partout : un mode clair/sombre, un système d'alertes (flash) qui se referment automatiquement, un planning en glisser-déposer avec zoom, et surtout une traduction complète français/anglais via un service dédié. C'est un vrai effort d'expérience utilisateur.

Mais pour une application dont la mission est la supervision, les manques sont les mêmes que ceux relevés ailleurs :

- **Pas de temps réel** : le tableau de bord ne se rafraîchit pas seul ; un lecteur tombé en panne n'apparaît qu'après rechargement manuel, sans alerte visuelle ni sonore. L'état UP/KO n'est même pas affiché.
- **Boutons et zones non fonctionnels** : l'espace commercial (recherche, bouton de lecture) n'est relié à aucune action, le panneau « Terminal » de l'admin est un cadre purement décoratif, et le bouton « Télécharger les logs » ne pointe vers aucune route. L'interruption d'urgence ne peut donc pas être déclenchée depuis l'interface, et il n'existe aucun bouton pour l'annuler.
- **Doublets d'écrans** : plusieurs versions de pages cohabitent (login vs login_v2, page testRasp de développement), ce qui rend l'interface difficile à maintenir et le parcours peu lisible.

- **Détail produit** : l'ajout de MP3 renvoie vers un service tiers externe de conversion, ce qui est peu professionnel pour un outil destiné à une entreprise cliente.

Note critère 4 : 12/20

Pistes : rafraîchissement automatique (polling ou WebSocket) avec pastilles d'état UP/KO et alerte visible ; câbler ou retirer les boutons inactifs ; un bouton d'arrêt immédiat pour l'annonce urgente ; ne conserver qu'une seule version par écran.

Critère 5 — Installation et déploiement

Bon point : contrairement à des projets fournissant un script .bat Windows, MusiQuali est réellement orienté Linux (scripts `becomeserver.sh` / `initialisationRaspberry.sh`, environnement virtuel sous python3.12, `ssh/rsync`), ce qui respecte l'exigence technique du cahier des charges. La base se crée toute seule au lancement, avec un compte admin par défaut, ce qui facilite la prise en main.

Le déploiement reste toutefois lourd et fragile :

- **Documentation d'installation manquante** : aucun README ni fichier `requirements.txt` n'apparaît dans les fichiers fournis, alors que le projet dépend de plusieurs bibliothèques non triviales (`bcrypt`, `mutagen`, `pygame`). Les dépendances ne sont donc ni listées ni figées.
- **Reste de gabarit non nettoyé** : le dépôt contient encore `ddl_img.py`, un script qui télécharge 905 images de Pokémon (hérité d'un modèle de cours) totalement étranger au projet symptôme d'un dépôt non assaini.
- **Configuration codée en dur** : l'URL `http://localhost:8000` est inscrite dans un template, tandis que les notes SSH évoquent d'autres ports (8080, tunnel 900x) l'histoire de déploiement est incohérente.
- **Lancement des lecteurs fragile** : la mise à jour pousse les fichiers puis exécute `RAS.py` par SSH avec un subprocess bloquant ; or `RAS.py` contient une boucle infinie, ce qui fige la requête web qui l'a lancée. La mise en route « branche et ça marche » voulue par le client n'est pas atteinte : elle suppose plusieurs étapes manuelles (configuration SSH en root, échange de clés, `rsync`).

Note critère 5 : 12/20

Pistes : ajouter un README pas-à-pas et un requirements.txt ; supprimer ddl_img.py et les restes de gabarit ; externaliser host/port dans une configuration ; lancer RAS.py en service non bloquant (systemd ou nohup) plutôt que par SSH synchrone.

Conclusion et décision

Critère	Note	Bar.
1 — Fonctionnalité / respect du besoin	12	/20
2 — Qualité du code	11	/20
3 — Gestion des données	9	/20
4 — Interface utilisateur	12	/20
5 — Installation & déploiement	12	/20
TOTAL	56	/100

Avec un total de 56/100, MusiQuali se révèle un projet honnête et, sur plusieurs aspects, plus abouti que d'autres dépôts analysés : il traite réellement le réseau multi-site, hache ses mots de passe, protège des injections SQL, propose une interface bilingue soignée et tourne sous Linux. Ces qualités sont réelles et nous tenons à les souligner.

Cependant, le cœur fonctionnel attendu par le client reste incomplet : l'interruption par message urgent pourtant centrale dans la consigne n'est pas implémentée dans le moteur de diffusion, la consignation centrale des logs n'est pas opérationnelle, et la supervision UP/KO se limite à un ping manuel sans alerte ni état persistant. À cela s'ajoutent une base exposée publiquement, un modèle de données non normalisé (relations en chaînes de caractères, données dupliquées dans le planning), des schémas multiples et incohérents, ainsi qu'un dépôt encombré de code mort, de doublons et même de conflits Git non résolus.

Reprendre MusiQuali nous obligerait donc à reconstruire le moteur de diffusion (canaux, messages urgents), à mettre en place la remontée et l'affichage des logs, à assainir le dépôt et à refondre le modèle de données un volume de travail qui, au final, dépasserait celui de la consolidation de notre propre projet, SoundStream, mieux noté et déjà aligné sur ces exigences.

Note de méthodologie : à l'image des autres dépôts, MusiQuali ne fournit pas de planification détaillée (Gantt avec tâches imbriquées, membres assignés et durées). Cette planification, demandée par l'encadrant, est précisément l'un des points sur lesquels notre équipe s'appuie pour la suite ; elle pourra être complétée une fois notre propre Gantt finalisé.

Décision finale : notre équipe décide de ne pas reprendre le projet MusiQuali pour la suite de la SAÉ, et de poursuivre sur la base de notre propre projet, SoundStream.